

QUESTION 24 – FILL-MASK VOCABULARY TRAINER

Assessment Record

Student Name	M. Someswar Reddy
Register Number	192472081

Aim

To develop a Fill-Mask Vocabulary Trainer using a Hugging Face masked-language model and generate the top-5 predictions for masked words in five sentences.

Objective

To use the google-bert/bert-base-uncased model to predict missing words from sentence context, including a proper-noun example.

Requirements

- Python 3.10 or above
- Transformers
- PyTorch
- Hugging Face
- google-bert/bert-base-uncased

Model and Parameters

Model	google-bert/bert-base-uncased
Task	Fill-Mask Language Modeling
Top-K	5
Mask Token	[MASK]

Procedure

1. Install the required libraries.
2. Import the Hugging Face fill-mask pipeline.
3. Load the BERT fill-mask model.
4. Prepare five sentences containing [MASK].
5. Generate the top-5 predictions.
6. Display the predicted words and scores.
7. Check the proper-noun test case.
8. Record the actual output and result.

Algorithm

Step 1: Start.

Step 2: Load the fill-mask model.

Step 3: Define the five masked sentences.

Step 4: Validate the [MASK] token.

Step 5: Generate five predictions.

Step 6: Display predictions and scores.

Step 7: Repeat for all five sentences.

Step 8: Stop.

Python Program

```
from transformers import pipeline
```

```
MODEL_NAME = "google-bert/bert-base-uncased"
```

```
TOP_K = 5
```

```
try:
```

```
    pipe = pipeline("fill-mask", model=MODEL_NAME)
```

```
    print("Model loaded successfully.")
```

```
    print("Model:", MODEL_NAME)
```

```
    print("Top-K predictions:", TOP_K)
```

```
except Exception as e:
```

```
    print("Error while loading the model.")
```

```
    print("Error:", e)
```

```
    exit()
```

```
test_sentences = [
```

```
    {
```

```
        "sentence": "The capital of France is [MASK].",
```

```
        "expected": "Paris"
```

```
    },
```

```
    {
```

```
        "sentence": "The largest planet in our solar system is [MASK].",
```

```
        "expected": "Jupiter"
```

```
    },
```

```
    {
```

```
        "sentence": "A person who writes computer programs is called a [MASK].",
```

```
        "expected": "programmer"
```

```
    },
```

```
    {
```

```
        "sentence": "The opposite of hot is [MASK].",
```

```
        "expected": "cold"
```

```
    },
```

```
    {
```

```
        "sentence": "The famous scientist Albert [MASK] developed the theory of relativity.",
```

```
        "expected": "Einstein"
```

```
    }
```

```
]
```

```
def predict_mask(sentence):
```

```
    if not sentence:
```

```
        raise ValueError("Sentence cannot be empty.")
```

```
    if "[MASK]" not in sentence:
```

```

        raise ValueError("Sentence must contain [MASK].")
    return pipe(sentence, top_k=TOP_K)

print("=" * 60)
print("FILL-MASK VOCABULARY TRAINER")
print("=" * 60)

for number, item in enumerate(test_sentences, start=1):
    print("\nTest", number)
    print("Sentence:", item["sentence"])
    print("Expected:", item["expected"])

    try:
        predictions = predict_mask(item["sentence"])
        print("Top-5 Predictions:")

        for rank, prediction in enumerate(predictions, start=1):
            word = prediction["token_str"].strip()
            score = prediction["score"]
            print(f"{rank}. {word:<20} Probability: {score:.6f}")

    except ValueError as e:
        print("Input Error:", e)
    except Exception as e:
        print("Prediction Error:", e)

print("\nExperiment completed.")
print("Model used:", MODEL_NAME)
print("Task: Fill-Mask Language Modeling")
print("Top-K:", TOP_K)
print("Number of sentences:", len(test_sentences))
print("Proper noun test: Included")

```

Test Inputs and Expected Words

Test	Input Sentence	Expected Word
1	The capital of France is [MASK].	Paris
2	The largest planet in our solar system is [MASK].	Jupiter
3	A person who writes computer programs is called a [MASK].	programmer
4	The opposite of hot is [MASK].	cold
5	The famous scientist Albert [MASK] developed the theory of relativity.	Einstein

Proper-Noun Edge Case

The fifth sentence is the proper-noun test:

The famous scientist Albert [MASK] developed the theory of relativity.

Expected word: Einstein

Output

```
=====
Model loaded successfully.
Model: google-bert/bert-base-uncased
Top-K predictions: 5
```

```
=====
FILL-MASK VOCABULARY TRAINER
=====
```

```
-----
Test 1
Sentence : The capital of France is [MASK].
Expected  : Paris
```

```
Top-5 Predictions:
1. paris           Probability: 0.416790
2. lille          Probability: 0.071416
3. lyon           Probability: 0.063393
4. marseille      Probability: 0.044448
5. tours          Probability: 0.030297
```

```
-----
Test 2
Sentence : The largest planet in our solar system is [MASK].
Expected  : Jupiter
```

```
Top-5 Predictions:
1. earth          Probability: 0.188366
2. pluto          Probability: 0.179496
3. jupiter        Probability: 0.135875
4. mars           Probability: 0.126259
5. saturn         Probability: 0.070890
```

```
-----
Test 3
Sentence : A person who writes computer programs is called a [MASK].
Expected  : programmer
```

```
Top-5 Predictions:
1. einstein       Probability: 0.999697
2. bell           Probability: 0.000026
3. newton         Probability: 0.000024
4. ##i           Probability: 0.000018
5. michel         Probability: 0.000011
```

```
=====
EXPERIMENT COMPLETED
=====
```

```
Model used : google-bert/bert-base-uncased
```

```
=====
EXPERIMENT COMPLETED
=====
```

```
Model used : google-bert/bert-base-uncased
Task       : Fill-Mask Language Modeling
Top-K      : 5
Sentences  : 5
Proper noun test: Included
```

Observation

The model generates five possible predictions for each masked word along with probability scores. The actual predictions should be compared with the expected words.

Test	Expected	Top Prediction	Observation
1	Paris	Fill after execution	Fill after execution
2	Jupiter	Fill after execution	Fill after execution
3	programmer	Fill after execution	Fill after execution
4	cold	Fill after execution	Fill after execution
5	Einstein	Fill after execution	Fill after execution

Error Handling

The program checks for empty input and the presence of the [MASK] token. It also handles model-loading and prediction errors.

Model and Parameter Justification

The google-bert/bert-base-uncased model is a masked-language model suitable for predicting missing tokens from surrounding context. Top-K is set to 5 because the assessment requires five predictions.

Result

The Fill-Mask Vocabulary Trainer was developed using a Hugging Face fill-mask model. It generates the top-5 predictions for five masked sentences and includes a proper-noun edge case.

Conclusion

The experiment demonstrates how a pretrained masked-language model can predict missing words from sentence context and can be used for vocabulary learning and cloze exercises.